

Grid Cell Predictions: Assessing factors driving marbled murrelet distribution and density in Puget Sound using hierarchical distance sampling

Code by S.J. Gillman

Required Packages

```
library(tidyverse)
library(nimble)
library(MCMCvis)
library(sf)
library(matrixStats)
```

Load Data

Required Files Previously Made or Provided

- Survey_Grid_Info_NEW.RData
- chain_1.rds
- chain_2.rds
- chain_3.rds
- nimConstants.rds
- nimData.rds

```
load("../DataAnalysis/DataFiles/Survey_Grid_Info_NEW.RData")
samples_1 <- readRDS("RESULTS/COMPRESSED/chain_1.rds")
samples_2 <- readRDS("RESULTS/COMPRESSED/chain_2.rds")
samples_3 <- readRDS("RESULTS/COMPRESSED/chain_3.rds")

nimConstants <- readRDS("RESULTS/COMPRESSED/nimConstants.rds")
nimData <- readRDS("RESULTS/COMPRESSED/nimData.rds")

post_pred <- as.matrix(rbind(samples_1, samples_2, samples_3))

colnames(nimData$DYNAMIC)
# alpha_sea[1] is PAR
# alpha_sea[2] is DO
# alpha_sea[3] is HQH
# alpha_sea[4] is GT
# alpha_sea[5] is DSHORE
# alpha_sea[6] is DSHORESQ

colnames(nimData$STAT_COVS)
# alpha_stat[1] is DEPTH
# alpha_stat[2] is BPI
```

```

# alpha_stat[3] is RIE
# alpha_stat[4] is EI-SAND
# alpha_stat[5] is EI-MIXF
# alpha_stat[6] is EI-BED
# alpha_stat[7] is EI-ART
# alpha_stat[8] is EI-GRAV
# alpha_stat[9] is MI
# alpha_stat[10] is OCN
# alpha_stat[11] is GRAVEL
# alpha_stat[12] is MUD
# alpha_stat[13] is SAND
# alpha_stat[14] is SANDYMUDD
# alpha_stat[15] is UMS
# alpha_stat[16] is FUS
# alpha_stat[17] is CUS
# alpha_stat[18] is COBBLE

```

Subset Posterior Samples

```

# For Monthly Estimates
coefs_month <- post_pred %>%
  as.data.frame() %>%
  dplyr::select(c(contains("eps_mon"),
                  contains("alpha_sea"),
                  contains("alpha_stat")))

# For Seasonal Estimates
coefs_seas <- post_pred %>%
  as.data.frame() %>%
  dplyr::select(c(contains("alpha0"),
                  contains("alpha_sea"),
                  contains("alpha_stat")))

```

Center and Scale Predictors

```

dynamic_scale <- survey_info %>%
  group_by(month) %>%
  summarise(par_mean = mean(par, na.rm=T),
            par_sd = sd(par, na.rm = T),
            do_mean = mean(do, na.rm=T),
            do_sd = sd(do, na.rm=T),
            gt_mean = mean(grand_range, na.rm=T),
            gt_sd = sd(grand_range, na.rm=T)) %>%
  ungroup()

dynamic_ranges <- survey_info %>%
  group_by(month) %>%
  summarise(par_min = min(par, na.rm=T),
            par_max = max(par, na.rm = T),

```

```

do_min = min(do, na.rm=T),
do_max = max(do, na.rm=T),
gt_min = min(grand_range, na.rm=T),
gt_max = max(grand_range, na.rm=T)) %>%
ungroup()

static_scale <- survey_info %>%
  summarise(hab_mean = mean(nest_hab, na.rm=T),
            hab_sd = sd(nest_hab, na.rm=T),
            shore_mean = mean(dist_shore, na.rm=T),
            shore_sd = sd(dist_shore, na.rm=T),
            depth_mean = mean(depth, na.rm=T),
            depth_sd = sd(depth, na.rm=T),
            bpi_mean = mean(bpi, na.rm=T),
            bpi_sd = sd(bpi, na.rm=T),
            rie_mean = mean(rie, na.rm=T),
            rie_sd = sd(rie, na.rm=T))

static_range <- survey_info %>%
  summarise(hab_min = min(nest_hab, na.rm=T),
            hab_max = max(nest_hab, na.rm=T),
            shore_min = min(dist_shore, na.rm=T),
            shore_max = max(dist_shore, na.rm=T),
            depth_min = min(depth, na.rm=T),
            depth_max = max(depth, na.rm=T),
            bpi_min = min(bpi, na.rm=T),
            bpi_max = max(bpi, na.rm=T),
            rie_min = min(rie, na.rm=T),
            rie_max = max(rie, na.rm=T))

static_range2 <- grid_info %>%
  summarise(hab_min = min(nest_hab, na.rm=T),
            hab_max = max(nest_hab, na.rm=T),
            shore_min = min(dist_shore, na.rm=T),
            shore_max = max(dist_shore, na.rm=T),
            depth_min = min(depth, na.rm=T),
            depth_max = max(depth, na.rm=T),
            bpi_min = min(bpi, na.rm=T),
            bpi_max = max(bpi, na.rm=T),
            rie_min = min(rie, na.rm=T),
            rie_max = max(rie, na.rm=T))

grid_substrate <- grid_info %>%
  distinct(ID, seafloor, shoretype) %>%
  mutate(dummy = 1) %>%
  pivot_wider(names_from = shoretype,
              values_from = dummy,
              values_fill = list(dummy = 0)) %>%

```

```

mutate(cmec_abb = ifelse(
  seafloor == "Coarse Unconsolidated Substrate", "CUS",
  ifelse(seafloor == "Fine Unconsolidated Substrate", "FUS",
  ifelse(seafloor == "Unconsolidated Mineral Substrate",
    "UMS", gsub(" ", "", seafloor)))) %>%
dplyr::select(-seafloor) %>%
mutate(dummy = 1) %>%
pivot_wider(names_from = cmec_abb,
  values_from = dummy,
  values_fill = list(dummy = 0)) %>%
select(c(ID,
  # shoreline EI-Mix_Coarse is reference
  `EI-Sand`,
  `EI-Mix_Fine`,
  `EI-Boulder`,
  `EI-Artificial`,
  `EI-Gravel`,
  MI,
  OCN,
  # substrate Muddy Sand is reference
  Gravel,
  Mud,
  Sand,
  SandyMud,
  UMS,
  FUS,
  CUS,
  Cobble))

grid_scale <- grid_info %>%
  rename(month = mon) %>%
  full_join(dynamic_scale, by = "month") %>%
  full_join(dynamic_ranges, by = "month") %>%
  # Ensure not above max or min
  mutate(par = ifelse(par > par_max, par_max,
    ifelse(par < par_min, par_min, par)),
    do = ifelse(do > do_max, do_max,
    ifelse(do < do_min, do_min, do)),
    grand_range = ifelse(grand_range > gt_max, gt_max,
    ifelse(grand_range < gt_min, gt_min, grand_range)),
    dist_shore = ifelse(dist_shore > static_range$shore_max,
    static_range$shore_max,
    ifelse(dist_shore < static_range$shore_min,
    static_range$shore_min, dist_shore)),
    nest_hab = ifelse(nest_hab > static_range$hab_max,
    static_range$hab_max,
    ifelse(nest_hab < static_range$hab_min,
    static_range$hab_min, nest_hab)),
    bpi = ifelse(bpi > static_range$bpi_max, static_range$bpi_max,
    ifelse(bpi < static_range$bpi_min,
    static_range$bpi_min, bpi)),
    rie = ifelse(rie > static_range$rie_max, static_range$rie_max,

```

```

        ifelse(rie < static_range$rie_min,
                static_range$rie_min, rie)),
depth = ifelse(depth > static_range$depth_max,
                static_range$depth_max,
                ifelse(depth < static_range$depth_min,
                        static_range$depth_min, depth))) %>%
mutate(par_scale = (par - par_mean)/par_sd,
       do_scale = (do - do_mean)/do_sd,
       hab_scale = (nest_hab - static_scale$hab_mean)/static_scale$hab_sd,
       gt_scale = (grand_range - gt_mean)/gt_sd,
       shore_scale = (dist_shore - static_scale$shore_mean)/
                     static_scale$shore_sd,
       shoreSQ_scale = shore_scale*shore_scale,
       depth_scale = (depth - static_scale$depth_mean)/
                     static_scale$depth_sd,
       bpi_scale = (bpi - static_scale$bpi_mean)/
                     static_scale$bpi_sd,
       rie_scale = (rie - static_scale$rie_mean)/
                     static_scale$rie_sd) %>%
## NAs will be filled with averages...
mutate(par_scale = ifelse(is.na(par_scale), mean(par_scale, na.rm=T),
                          par_scale),
       gt_scale = ifelse(is.na(gt_scale), mean(gt_scale, na.rm=T),
                          gt_scale),
       do_scale = ifelse(is.na(do_scale), mean(do_scale, na.rm=T),
                          do_scale)) %>%
select(c(ID, numid, seasNum, month, mNum, year, myNum,
        par_scale, do_scale, hab_scale,
        gt_scale, shore_scale, shoreSQ_scale, depth_scale,
        bpi_scale, rie_scale, ave_gs)) %>%
full_join(grid_substrate, by = "ID") %>%
full_join(grid_info %>%
          distinct(ID, area), by = "ID") %>%
arrange(year, mNum, numid)

sapply(grid_scale, function(x) sum(is.na(x)))

total_area <- grid_info %>%
  distinct(ID, area) %>%
  summarise(totarea = sum(area)) %>%
  pull(totarea)

total_area

```

Make Functions for Density Predictions

Month-Year

```

predict_density <- function(target_month, target_year,
                             grid_scale, coefs_month) {

```

```

# 6 dynamic environmental variables (change by season index)
dynamic_covs <- c("par_scale", "do_scale", "hab_scale",
                  "gt_scale", "shore_scale", "shoreSQ_scale")

# 18 static environmental variables (constant across seasons)
static_covs <- c("depth_scale", "bpi_scale", "rie_scale",
                 "EI-Sand", "EI-Mix_Fine",
                 "EI-Boulder", "EI-Artificial", "EI-Gravel",
                 "MI", "OCN", "Gravel",
                 "Mud", "Sand", "SandyMud", "UMS",
                 "FUS", "CUS", "Cobble")

# Subset grid data for this specific month-year chunk
df_sub <- grid_scale %>%
  filter(month == target_month, year == target_year) %>%
  arrange(numid)

G <- nrow(df_sub)
I <- nrow(coefs_month)
if(G == 0) return(NULL) # Skip if no grid data matches

# Convert grid covariates into design matrices
DYNAMIC <- as.matrix(df_sub[, dynamic_covs])
STAT_COVS <- as.matrix(df_sub[, static_covs])

# Pull model indices unique to this month-year chunk
mID <- unique(df_sub$mNum)
sID <- unique(df_sub$seasNum)
myID <- unique(df_sub$myNum)

# Extract posterior parameters using explicit naming
eps_mon <- coefs_month[[paste0("eps_mon[", mID, "]")]]
gs_ave <- df_sub$ave_gs

sea_cols <- paste0("alpha_sea[", sID, ", ", 1:6, "]")
alpha_sea <- as.matrix(coefs_month[, sea_cols])

stat_cols <- paste0("alpha_stat[", 1:18, "]")
alpha_stat <- as.matrix(coefs_month[, stat_cols])

# Multiply
dynamic_coefs <- DYNAMIC %*% t(alpha_sea)
static_coefs <- STAT_COVS %*% t(alpha_stat)

# Combine components into log lambda space
log_group <- sweep(dynamic_coefs + static_coefs, 2, eps_mon, `+`)

rm(dynamic_coefs, static_coefs)

# Group Size
est_lambda <- exp(log_group) * gs_ave
rm(log_group)

```

```

# Summarize across posterior draws
quants <- rowQuantiles(est_lambda, probs = c(0.025, 0.25, 0.5, 0.75, 0.975),
                      na.rm = TRUE)

summary_df <- data.frame(
  numid = df_sub$numid,
  month = target_month,
  year = target_year,
  AVE = rowMeans(est_lambda, na.rm = TRUE),
  MED = quants[, "50%"],
  LCI = quants[, "2.5%"],
  Q25 = quants[, "25%"],
  Q75 = quants[, "75%"],
  UCI = quants[, "97.5%"],
  SD = rowSds(est_lambda, na.rm = TRUE)) %>%
  mutate(CV = SD / AVE, IQR = Q75 - Q25)

return(list(summary = summary_df,
            raw_posterior = est_lambda,
            metadata = df_sub[, c("numid", "month", "year", "area")]))
}

```

Average Month

```

month_ave <- function(target_month, mo_yr_grid, all_months) {

  # Find which list positions match your target month across all years
  matching_indices <- which(mo_yr_grid$month == target_month)
  n_years <- length(matching_indices)
  if(n_years == 0) return(NULL)

  # Grid dimensions
  G <- nrow(all_months[[matching_indices[1]]$raw_posterior)
  I <- ncol(all_months[[matching_indices[1]]$raw_posterior)

  # Pre-allocating prevents RAM fragmentation and duplication crashes
  total_cols <- I * n_years
  pooled_posterior <- matrix(0, nrow = G, ncol = total_cols)

  # Fill matrix directly by index slice
  for (k in seq_along(matching_indices)) {
    idx <- matching_indices[k]
    col_start <- (k - 1) * I + 1
    col_end <- k * I
    pooled_posterior[, col_start:col_end] <- all_months[[idx]]$raw_posterior
  }

  # Calculate quantiles in C (matrixStats)
  quants <- rowQuantiles(pooled_posterior,
                        probs = c(0.025, 0.25, 0.5, 0.75, 0.975),
                        na.rm = TRUE)
}

```

```

# Construct long-term summary dataframe
long_term_summary <- data.frame(
  numid = all_months[[matching_indices[1]]]$metadata$numid,
  month = target_month,
  AVE = rowMeans(pooled_posterior, na.rm = TRUE),
  MED = quantiles[, "50%"],
  LCI = quantiles[, "2.5%"],
  Q25 = quantiles[, "25%"],
  Q75 = quantiles[, "75%"],
  UCI = quantiles[, "97.5%"],
  SD = rowSds(pooled_posterior, na.rm = TRUE)) %>%
  mutate(CV = SD / AVE, IQR = Q75 - Q25)

# Free pooled memory immediately
rm(pooled_posterior, quantiles)
gc(verbose = FALSE)

return(long_term_summary)
}

```

Global Month-Year

```

overall_month_year <- function(target_month, target_year, all_months) {

  # Finds exact list element matching target month & year directly
  match_idx <- Fast_match_idx <- NULL
  for (i in seq_along(all_months)) {
    meta <- all_months[[i]]$metadata
    if (meta$month[1] == target_month && meta$year[1] == target_year) {
      match_idx <- i
      break
    }
  }

  if (is.null(match_idx)) return(NULL)

  # Extract components
  raw_mat <- all_months[[match_idx]]$raw_posterior
  cell_areas <- as.numeric(all_months[[match_idx]]$metadata$area)

  # Crossprod computes t(cell_areas) %*% raw_mat directly
  total_abund <- as.numeric(crossprod(cell_areas, raw_mat))

  # Compute Abundance Summary
  quantiles_abund <- quantile(total_abund,
    probs = c(0.025, 0.25, 0.5, 0.75, 0.975),
    na.rm = TRUE)

  abund_summary <- data.frame(
    month = target_month,
    year = target_year,
    AVE = mean(total_abund, na.rm = TRUE),

```



```

MED = quants_abund["50%"],
LCI = quants_abund["2.5%"],
Q25 = quants_abund["25%"],
Q75 = quants_abund["75%"],
UCI = quants_abund["97.5%"],
SD = sd(total_abund, na.rm = TRUE),
row.names = NULL) %>% mutate(CV = SD / AVE, IQR = Q75 - Q25)

# Derive overall density summary by dividing abundance summary by total_area scalar
total_area <- sum(cell_areas)

overall_summary <- abund_summary %>%
  mutate(AVE = AVE / total_area,
    MED = MED / total_area,
    LCI = LCI / total_area,
    Q25 = Q25 / total_area,
    Q75 = Q75 / total_area,
    UCI = UCI / total_area,
    SD = SD / total_area,
    IQR = IQR / total_area)

return(list(densities = overall_summary, abundance = abund_summary))
}

```

Global Month

```

month_ave_global <- function(target_month, mo_yr_grid, all_months) {

  matching_indices <- which(mo_yr_grid$month == target_month)
  if (length(matching_indices) == 0) return(NULL)

  # Extract matrices for target month across all years
  raw_matrices <- map(matching_indices, ~ all_months[.[.x]]$raw_posterior)

  # Combine horizontally
  pooled_posterior <- do.call(cbind, raw_matrices)

  # Clean up list copy from RAM
  rm(raw_matrices)

  # Calculate Mean directly from matrix (no flattening required)
  ave_val <- mean(pooled_posterior, na.rm = TRUE)

  # Calculate Standard Deviation efficiently
  sd_val <- sd(as.vector(pooled_posterior), na.rm = TRUE)

  # Compute Quantiles directly
  quants <- quantile(pooled_posterior, probs = c(0.025, 0.25,
    0.5, 0.75, 0.975),
    na.rm = TRUE)

  # Free pooled matrix from memory
}

```

```

rm(pooled_posterior)
gc(verbose = FALSE)

# Build clean summary dataframe
long_term_summary <- data.frame(
  month = target_month,
  AVE = ave_val,
  MED = quantiles["50%"],
  LCI = quantiles["2.5%"],
  Q25 = quantiles["25%"],
  Q75 = quantiles["75%"],
  UCI = quantiles["97.5%"],
  SD = sd_val,
  row.names = NULL) %>%
  mutate(CV = SD / AVE, IQR = Q75 - Q25)

return(long_term_summary)
}

```

Season-Year

```

predict_density_SEASON <- function(target_season, target_year,
                                   grid_scale, coefs_seas) {

  # 6 dynamic environmental variables
  dynamic_covs <- c("par_scale", "do_scale", "hab_scale",
                    "gt_scale", "shore_scale", "shoreSQ_scale")

  # 18 static environmental variables
  static_covs <- c("depth_scale", "bpi_scale", "rie_scale",
                  "EI-Sand", "EI-Mix_Fine",
                  "EI-Boulder", "EI-Artificial", "EI-Gravel",
                  "MI", "OCN", "Gravel",
                  "Mud", "Sand", "SandyMud", "UMS",
                  "FUS", "CUS", "Cobble")

  # Subset grid data for this specific season chunk
  df_sub <- grid_scale %>%
    filter(seasNum == target_season, year == target_year) %>%
    arrange(numid)

  if (nrow(df_sub) == 0) return(NULL)

  sID <- target_season
  gID <- sort(unique(df_sub$numid))

  # Extract parameters once
  alpha0 <- coefs_seas[[paste0("alpha0", sID, " ")]]
  sea_cols <- paste0("alpha_sea", sID, " ", "1:6, ")
  alpha_sea <- as.matrix(coefs_seas[, sea_cols])
  stat_cols <- paste0("alpha_stat", "1:18, ")
  alpha_stat <- as.matrix(coefs_seas[, stat_cols])
}

```

```

# Split by myNum instead of running filter() in a loop
month_list <- split(df_sub, df_sub$myNum)
month_matrices <- vector("list", length(month_list))

for (m in seq_along(month_list)) {
  df_month <- month_list[[m]]
  if (nrow(df_month) == 0) next

  DYNAMIC <- as.matrix(df_month[, dynamic_covs])
  STAT_COVS <- as.matrix(df_month[, static_covs])
  gs_ave <- df_month$ave_gs

  # linear algebra
  dynamic_coefs <- DYNAMIC %*% t(alpha_sea)
  static_coefs <- STAT_COVS %*% t(alpha_stat)

  # Sweep alpha0 across columns
  log_group <- sweep(dynamic_coefs + static_coefs, 2, alpha0, `+`)

  # Clean up intermediate matrices
  rm(dynamic_coefs, static_coefs)

  # Exponentiate and multiply gs_ave down rows directly
  month_matrices[[m]] <- exp(log_group) * gs_ave
  rm(log_group)
}

# Drop NULL/empty list elements if any months were empty
month_matrices <- compact(month_matrices)
if (length(month_matrices) == 0) return(NULL)

# Pool all months in this season horizontally
pooled_posterior <- do.call(cbind, month_matrices)
rm(month_matrices)

# Calculate 50% median along with quantiles in a single C pass
probs <- c(0.025, 0.25, 0.5, 0.75, 0.975)
quants <- rowQuantiles(pooled_posterior, probs = probs, na.rm = TRUE)

summary_df <- data.frame(
  numid = gID,
  seasNum = target_season,
  year = target_year,
  AVE = rowMeans(pooled_posterior, na.rm = TRUE),
  MED = quants[, "50%"],
  LCI = quants[, "2.5%"],
  Q25 = quants[, "25%"],
  Q75 = quants[, "75%"],
  UCI = quants[, "97.5%"],
  SD = rowSds(pooled_posterior, na.rm = TRUE)) %>%
  mutate(CV = SD / AVE, IQR = Q75 - Q25)

return(list(summary = summary_df,

```

```

    raw_posterior = pooled_posterior,
    metadata = data.frame(numid = gID,
                          seasNum = target_season,
                          year = target_year)))
}

```

Average Season

```

season_ave <- function(target_season, sea_yr_grid,
                      all_seasons, chunk_size = 100) {

  # Extract matching indices
  matching_indices <- which(sea_yr_grid$seasNum == target_season)
  if (length(matching_indices) == 0) return(NULL)

  sub_seasons <- all_seasons[matching_indices]

  # Extract baseline grid cell IDs
  first_ids <- sub_seasons[[1]]$metadata$numid
  G <- length(first_ids)

  # Setup chunk iteration parameters
  chunk_starts <- seq(1, G, by = chunk_size)
  results_list <- vector("list", length(chunk_starts))

  # Process grid in small chunks
  for (c in seq_along(chunk_starts)) {
    idx_start <- chunk_starts[c]
    idx_end <- min(idx_start + chunk_size - 1, G)
    chunk_gids <- first_ids[idx_start:idx_end]

    # Extract rows idx_start:idx_end from each year and combine horizontally
    chunk_mats <- lapply(sub_seasons, function(x) x$raw_posterior[idx_start:idx_end, ,
                                                                    drop = FALSE])

    pooled_chunk <- do.call(cbind, chunk_mats)
    rm(chunk_mats) # Clean up temporary list reference

    # Compute quantiles
    probs <- c(0.025, 0.25, 0.50, 0.75, 0.975)
    quants <- rowQuantiles(pooled_chunk, probs = probs, na.rm = TRUE)

    results_list[[c]] <- data.frame(
      numid = chunk_gids,
      seasNum = target_season,
      AVE = rowMeans(pooled_chunk, na.rm = TRUE),
      MED = quants[, "50%"],
      LCI = quants[, "2.5%"],
      Q25 = quants[, "25%"],
      Q75 = quants[, "75%"],
      UCI = quants[, "97.5%"],
      SD = rowSds(pooled_chunk, na.rm = TRUE))
  }
}

```

```

    # Free chunk buffer before next loop iteration
    rm(pooled_chunk)
    gc(verbose = FALSE)
  }

  # Combine chunk results into a single clean data frame
  long_term_summary <- bind_rows(results_list) %>%
    mutate(CV = SD / AVE, IQR = Q75 - Q25)

  return(long_term_summary)
}

```

Prediction Monthly

Month-Year

```

mo_yr_grid <- grid_scale %>% distinct(month, year)

# Run the matrix prediction across all combinations
all_months <- purrr::map2(mo_yr_grid$month, mo_yr_grid$year, function(m, y) {
  message(paste("Predicting Month-Year:", m, y))
  predict_density(m, y, grid_scale, coefs_month)
})

# Combine all
predicted_month_years <- map_df(all_months, ~ .x$summary)

gc()

```

```

saveRDS(predicted_month_years, file = "RESULTS/predicted_month_years.rds")

```

Monthly Average

```

unique_months <- unique(mo_yr_grid$month)

Monthly_averages <- map_df(unique_months, function(m) {
  message(paste("Calculating Average for:", m))
  month_ave(target_month = m, mo_yr_grid = mo_yr_grid, all_months = all_months)
})

gc()

```

```

saveRDS(Monthly_averages, file = "RESULTS/predicted_months.rds")

```

Global Month-Year Average

```

month_year_global <- purrr::map2(mo_yr_grid$month,
                                mo_yr_grid$year, function(m, y) {
  message(paste("Predicting Month-Year:", m, y))
  overall_month_year(m, y, all_months)
})

gc()

my_global_df <- map_df(month_year_global, ~ .x$densities)
myA_global_df <- map_df(month_year_global, ~ .x$abundance)
saveRDS(my_global_df, file = "RESULTS/predicted_month_year_global.rds")

saveRDS(myA_global_df, file = "RESULTS/predictedA_month_year_global.rds")

```

Global Monthly Average

```

Monthly_global <- map_df(unique_months, function(m) {
  message(paste("Calculating Average for:", m))
  month_ave_global(target_month = m, mo_yr_grid = mo_yr_grid,
                  all_months = all_months)
})

saveRDS(Monthly_global, file = "RESULTS/predicted_months_global.rds")

rm(all_months)
rm(samples_1)
rm(samples_2)
rm(samples_3)
rm(coefs_month)
rm(post_pred)
gc()

```

Prediction Seasonal

Season-Year

```

sea_yr_grid <- grid_scale %>% distinct(seasNum, year)

# Run the matrix prediction across all combinations
all_seasons <- purrr::map2(sea_yr_grid$seasNum,
                          sea_yr_grid$year, function(s, y) {
  message(paste("Predicting Season-Year:", s, y))
  predict_density_SEASON(s, y, grid_scale, coefs_seas)
})

# Combine all
predicted_season_years <- map_df(all_seasons, ~ .x$summary)

```

```
saveRDS(predicted_season_years, file = "RESULTS/predicted_season_years.rds")
```

Seasonal Average

```
Breeding_averages <- season_ave(target_season = 1,  
                                sea_yr_grid = sea_yr_grid,  
                                all_seasons = all_seasons, chunk_size = 100)  
gc()  
NBreeding_averages <- season_ave(target_season = 2,  
                                sea_yr_grid = sea_yr_grid,  
                                all_seasons = all_seasons, chunk_size = 100)
```

```
saveRDS(Breeding_averages, file = "RESULTS/predicted_seasonsB.rds")  
saveRDS(NBreeding_averages, file = "RESULTS/predicted_seasonsNB.rds")
```